2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C) | 978-1-6654-7836-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/QRS-C55045.2021.00096

Curious SDN for network attack mitigation

Mikhail Zolotukhin
Faculty of Information Technology
University of Jyväskylä
Jyväskylä, Finland
e-mail: mizolotu@jyu.fi

Timo Hämäläinen
Faculty of Information Technology
University of Jyväskylä
Jyväskylä, Finland
email: timoh@jyu.fi

Riku Immonen
Faculty of Information Technology
University of Jyväskylä
Jyväskylä, Finland
e-mail: rijuimmo@jyu.fi

***Abstract*—The increasing number of connected IoT devices in recent years has led to a significant growth in the volume of cyber attack instances. Since the IoT devices include home automation sensors, medical equipment, vehicular sensors, nuclear reactors and life-critical real-time sensing devices, lack of security in IoT can pose a risk to human lives. In this study, we focus on the problem of attack detection and mitigation with the help of recently emerging reinforcement machine learning which has already demonstrated excellent suitability for several cyber security applications. In particular, we are implementing an intelligent agent which manipulates network security policies depending on the current state of the environment. These manipulations include pushing software-defined networking flows to the network controller as well as adjusting detection sensitivity of security appliances used for intrusion and anomaly detection. To encourage the environment exploration, the agent is provided with an intrinsic curiosity reward signal based on how hard it is for the agent to predict the consequences of its own actions. We evaluate the resulting framework against several attack scenarios using realistic network traffic datasets and the simulation results confirmed the viability of the approach proposed.**

***Index Terms*—IoT, network security, deep learning, reinforcement learning, SDN, NFV**

I. Introduction

Recently emerging deep reinforcement learning (RL) is a machine learning (ML) paradigm in which software agents and machines automatically determine the ideal behavior within a specific context by continually making value judgments to select good actions over bad [1]. RL can model an autonomous agent to take sequential actions optimally without or with limited prior knowledge of the environment [2]. RL is believed to be suitable for designing artificial general intelligence systems which would be able to successfully perform any intellectual task that a human being can [3]. However, despite being a promising approach in theory, deep RL has thus far not seen wide adoption in the industry due to several practical obstacles. Probably, the biggest obstacle of applying reinforcement learning algorithms in real-life environments is its low sample efficiency, even though recent progress in developing better RL algorithms has made a big step towards overcoming this problem [4], [19]. As a result, industrial domains which benefit from employing RL are characterized by either tremendous amounts of data constantly generated in real time which can be used to train RL agents online or an ability to design a realistic simulation software to emulate the corresponding real world environment such that the RL agents can be trained in simulations. For this reason, RL is flourishing in finance and e-commerce exploiting available big data for algorithmic trading [5] and producing user recommendations [6] as well as robotics [7] and autonomous driving [8] for which many robust physics engines and sophisticated simulators have been developed.

RL also demonstrates excellent suitability for cyber security applications where attacks are increasingly sophisticated, rapid, and ubiquitous [9]. The sample efficiency problem can be partially solved by training the algorithms in a parallel fashion by emulating the real network environments with the help of cutting-edge cloud computing technologies including software-defined networking (SDN) and network function virtualization (NFV). SDN allows for centralized control of the network behavior enabling quick reactions to security threats, granular traffic filtering, and dynamic security policies deployment. SDN coupled with NFV extends conventional security mechanisms of a network system such as authentication and authorization, traffic filtering and firewalls, encryption protocols and anomaly-based detection with traffic isolation, centralized visibility, dynamic flow control, host and routing obfuscation, and security network programmability. In such a system, an RL-based agent would reside on top of the SDN and NFV controllers and manipulate network security policies depending on the current state of the environment by launching additional security functions, redirecting or mirroring certain network traffic, changing link capacities or blocking suspicious connections. Recent studies propose employing state-of-the-art RL algorithms with SDN and NFV for intelligent flow routing to maximize overall network throughput [10] and minimize latency [11], optimal deployment of virtual network functions (VNFs) to minimize placement related costs [12] as well as dynamic service function chaining (SFC) to maximize VNF resource utilization [13] and minimize processing delay [14]. In [15], [16], we contributed to this field of study by applying various reinforcement learning algorithms for mitigating real attacks in SDN-enabled network environments by redirecting certain network traffic between multiple virtual security appliances.

Another major obstacle to employing the RL approach is related to reward function specification. Shaping a reward

This paper is partially based on the results obtained during the project which has received funding from the European Union's Horizon 2020 research and innovation programme under the NGI_TRUST grant agreement no 825618.

2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion (QRS-C)

2693-9371/21/$31.00 ©2021 IEEE
DOI 10.1109/QRS-C55045.2021.00096

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:06 UTC from IEEE Xplore. Restrictions apply.

function in such a way that an agent can learn from it often requires considerable manual effort. Furthermore, in many real-world scenarios, rewards extrinsic to the agent are extremely sparse. In this case, the agent receives reinforcement for updating its policy only if it succeeds in reaching a certain goal state. Curiosity is an intrinsic reward that is equal to the error of the agent to predict the next state given the current state and the action taken [17]. Curiosity pushes the agent to favor transitions with high prediction error which are higher in areas where the agent has spent less time. As a result this intrinsic reward signal encourages the agent to explore the environment when looking for an optimal strategy.

In this study, we focus on intelligent attack detection and mitigation using the RL approach with the curiosity-based reward function. In particular, the RL agent processes traffic flowing through switches as well as log reports from security appliances deployed and changes the network security policy in the most optimal way by reconfiguring the security appliances, redirecting certain slices of network traffic or blocking suspicious connections. The agent takes its decision after observing features extracted from network traffic as well as alerts generated by security middle boxes. The rest of the document is organized as follows. Theoretical background is provided in Section II. Section III highlights implementation details of the framework proposed. Section IV presents simulation results. Section V concludes the report and outlines future work.

II. Theoretical Background

A. Advantage Actor-Critic

Many state-of-the-art RL algorithms rely on actor-critic architecture in which the actor selects an action a given state s according to its policy $\pi_{\theta_P}(a|s)$, whereas the critic estimates value $v_{\theta_V}(s)$ of being at that state. Here θ_P and θ_V are parameters of policy and value neural networks respectively. In particular, advantage actor-critic (A2C) algorithm uses value function to estimate advantage of being at each state $A(s_t, a_t) = r_{t+1} + \gamma v_{\theta_V}(s_{t+1}) - v_{\theta_V}(s_t)$ [18]. Here γ is a reward discount factor. Subtracting the value function from the cumulative reward reduces variance and increases stability. The loss function in this case can be calculated as the weighted sum of the policy and value losses:

$$\mathcal{L} = \mathcal{L}_\pi + \alpha \mathcal{L}_v, \tag{1}$$

where policy loss can be obtained as $\mathcal{L}_\pi = \mathbb{E}_{\pi_{\theta_P}}\left(-\sum_t (\log\left(\pi_{\theta_P}(a_t|s_t)\right) A(s_t, a_t)) + \delta H(\pi_{\theta_P}(\cdot|s_t))\right)$ and value loss is simply the sum of the squared advantage values $\mathcal{L}_v = \sum_t A(s_t, a_t)^2$. Here the entropy H regularization term is included into the objective function to encourage exploration, parameters α and δ regulate the importance of the loss function components.

Trust region policy optimization (TRPO) algorithm which relies on minimizing a certain surrogate objective function that guarantees policy improvement with non-trivial step sizes allows one to improve stability of the learning even further [19]. This algorithm uses average Kullback–Leibler divergence between the old policy and updated policy as a measurement for a region around the current policy parameters within which they trust the model to be an adequate representation of the objective function, and then chooses the step to be the approximate minimizer of the model in this region. Even though TRPO has achieved great and consistent high performance, its implementation can be extremely complicated and resource-hungry. The current state-of-the-art algorithm proximal policy optimization (PPO) attempts to reduce the complexity of TRPO implementation and computation by tracing the impact of the actions with a ratio between the probability of action under current policy divided by the probability of the action under previous policy and artificially clipping this value in order to avoid having too large policy update [4].

B. Intrinsic Curiosity Module

To further encourage the exploration, the objective function can be extended with an intrinsic reward signal based on how hard it is for the agent to predict the consequences of its own actions, which is usually referred as curiosity [17]. This reward signal can be obtained with the help of two following functions $f(\hat{s}_t, a_t)$ and $g(\hat{s}_t, \hat{s}_{t+1})$, the former of which is used to predict the feature encoding $\hat{s}_{t+1}$ of the state given the encoded state $\hat{s}_t$ and action a_t, whereas the latter predicts action a_t given encoded states $\hat{s}_t$ and $\hat{s}_{t+1}$. Representing these functions as neural networks with parameters θ_F and θ_I respectively, the loss function can be formulated as follows:

$$\mathcal{L} = \lambda(\mathcal{L}_{\pi_c} + \alpha \mathcal{L}_v) + \beta \mathcal{L}_\mathcal{F} + (1-\beta)\mathcal{L}_\mathcal{I}, \tag{2}$$

where policy loss now accounts for the agent's curiosity $L_{\pi_c} = \mathbb{E}_{\pi_{\theta_P}}\left(-\sum_t (\log\left(\pi_{\theta_P}(a_t|s_t)\right) A(s_t, a_t)) + \delta H(\pi_{\theta_P}(\cdot|s_t))\right) - \eta \sum_t ||f_{\theta_F}(\hat{s}_t, a_t) - \hat{s}_{t+1}||^2$, forward loss $\mathcal{L}_F = \sum_t ||f_{\theta_F}(\hat{s}_t, a_t) - \hat{s}_{t+1}||^2$, inverse loss $\mathcal{L}_I = \sum_t ||g_{\theta_I}(\hat{s}_t, \hat{s}_{t+1}) - a_t||^2$, whereas η, λ and β are hyperparameters.

III. Framework Implementation

In order to evaluate the RL approach proposed we have developed an environment which allows an RL agent to control the network security policy by pushing SDN flows and reconfiguring security appliances as shown in Figure 1. In order to train an intelligent agent in a reasonable amount of time, the training process can be carried out in several environments in parallel. For this reason, we build our training environment as a network of several virtual machines (VMs) running in a hypervisor. The main purpose of the SDN controller in the resulting framework is to transform the security intent of the RL agent to SDN flows and push them to the switches. In our implementation, we use OpenDayLight which is one of the most featured controllers that are able to run on different platforms. We develop an external application that uses OpenDayLight's restful APIs to find existing tables and flows on a switch, push a new flow to a certain table, and delete a flow from a table. These SDN flows allow us to manipulate network security policy by redirecting certain traffic or blocking suspicious connections.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:06 UTC from IEEE Xplore. Restrictions apply.

We implement our own security middle boxes in order to classify network packets with machine learning models. Once a network packet is captured by an interface of such a VNF, the packet content is parsed and the most relevant features are extracted from it using Pypacker which is the fastest and simplest packet manipulation library for Python. Next, the packet features are combined into flows. For each flow, a feature vector is calculated. The features include flow duration, number and size of packets, data rate, packet inter-arrival times and many others [20]. The resulting sequences of feature vectors are then used to classify the traffic as either legitimate or malicious using a Tensorflow machine learning model. In particular, we first train three supervised binary classifiers each with different type of the input layer, namely fully-connected (MLP), 1-dimensional convolution (CNN) and recurrent neural network (RNN) based on long short-term memory. In addition we train three unsupervised models for anomaly detection, which include autoencoder (AEN), self-organizing map (SOM) and bidirectional generative adversarial network (BGN). The attack detection performance evaluation of the aforementioned models can be found in Figure 5.

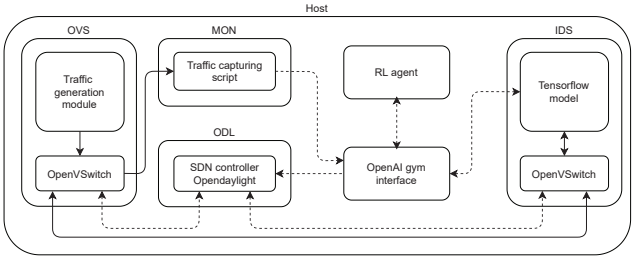


Fig. 1: The environment for training RL agent implemented. Network traffic flows and commands are shown by solid and dashed lines respectively.

We use OpenAI gym [21] to implement the frontend for the training environment. The RL agent is implemented using OpenAI baselines [22]. The training process is divided into episodes, each of which lasts a certain fixed amount of time steps. During each such episode, we simulate an attack scenario by generating corresponding malicious traffic flows mixed with legitimate ones. The RL agent observes the current network state and attempts to mitigate the threat by redirecting network traffic flows and reconfiguring virtual security appliances. The network state includes number of flows, packets and bytes sent between a pair of subnets via certain application as well as number of alerts generated by security functions for each such pair. The agent is capable of either redirecting a slice of the traffic into one of the security functions available or dropping the corresponding connections via pushing corresponding SDN flows. Furthermore, the agent is able to switch detection parameters of the security functions which include machine learning model type and sensitivity, which is essentially a threshold used to classify a value returned by each machine learning model as either corresponding to a normal traffic flow or malicious one. Finally, the reward for each action is calculated as the sum of percentage of the legitimate traffic passed and percentage of the malicious traffic blocked. The resulting environment implementation can be found in our project repository on Github [23].

IV. Simulation Results

To evaluate the framework proposed to mitigate intrusions we use network packet captures from CICIDS2018 dataset [20]. It contains 560 Gb of traffic generated during 10 days by 470 machines. The dataset in addition to benign samples includes the following attacks: DDoS attacks using GoldenEye, Slowloris, SlowHTTPTest and Hulk, botnet attacks which are presented as communication between an infected machine and a C2 server and Web application brute-force attacks including password guessing attempts, XSS and SQL injections. We first use the resulting datasets to train our machine learning models to classify known malicious flows as well as anomalies.

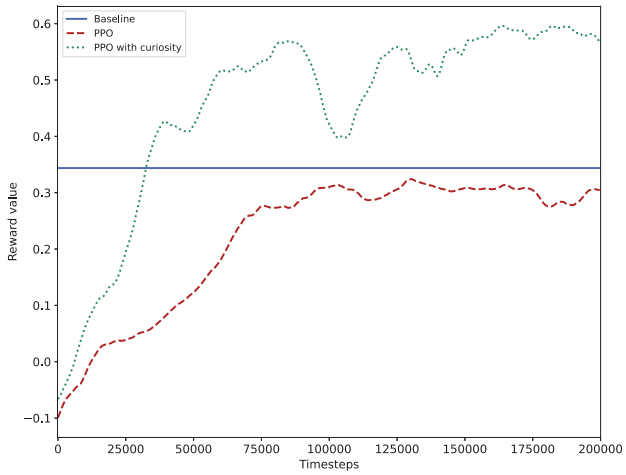


Fig. 2: Reward function during the training in case of DDoS attack mitigation.

We train the RL agent using PPO algorithm with and without the intrinsic curiosity signal. The network architecture consists of 1 one-dimensional convolutional layer with 512 filters shared by both the policy and value networks followed by two separate streams each of which consists of two fully-connected layers of 256 neurons. Both forward and inverse networks of the curiosity module consist of two fully-connected layers of 256 neurons. Values of the hyperparameters mentioned in Section II are the following: $\alpha = 0.5$, $\delta = 0.01$, $\gamma = 0.99$, $\eta = 0.01$, $\lambda = 0.1$, and $\beta = 0.2$. We run experiments in 4 copies of the environment in parallel each of which includes 2 security VNFs. During each episode, the agent takes 64 steps in each environment. As a baseline, we use a static network security policy which includes selecting two most accurate models with the best thresholds, redirecting all the traffic through the corresponding VNFs and blocking all the connections marked as malicious by at least one of the model[1].

[1]It is worth noticing that the static policy which only blocks connections marked as malicious by both models provides for slightly lower reward values.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:06 UTC from IEEE Xplore. Restrictions apply.

We calculate the reward for the baseline policy by running 10 tests and calculating the average value.

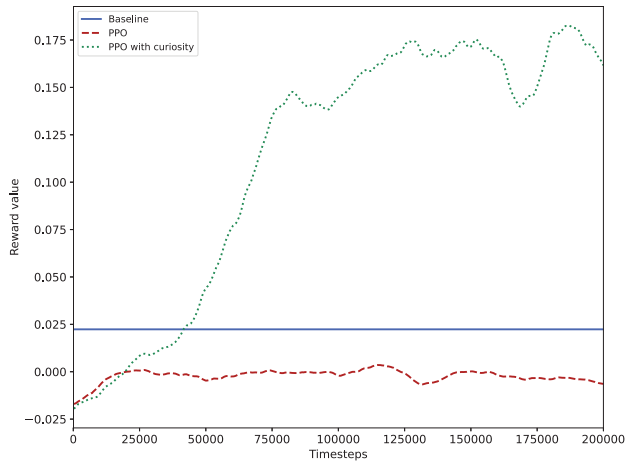


Fig. 3: Reward function during the training in case of botnet attack mitigation.

Figures 2, 3 and 4 show the moving-average evolution of the reward function throughout the training episodes for two aforementioned algorithms compared to the baseline strategy against three different attacks. As one can notice, in case of DDoS and botnet attacks, the curiosity-driven agent is able to outperform both the baseline and PPO agents. The key reason why this agent outperforms the baseline policy is lower drop rates for legitimate connections (see Figure 6). Surprisingly, PPO with entropy-based exploration shows quite poor results in the first two attack scenarios. However, in the third case, PPO reaches the baseline level and even starts slightly outperforming it. On contrary, the curiosity-driven agent fails in this scenario being unable to train accurately its forward and inverse models as the sum of these two losses stays high during the training and the reward gets stuck near zero.

V. Conclusion

In this study, we focused on intelligent attack detection and mitigation using the RL approach with the curiosity-based reward function. We designed a virtualized network environment which allows the RL agent to push SDN flows and reconfigure security appliances. We employed this environment to evaluate the attack mitigation approach proposed using realistic network traffic data. The simulation results proved the viability of the approach proposed for attack mitigation in SDN. In the future, we are planning to continue the research by implementing and evaluating various unsupervised reinforcement learning algorithms in different attack scenarios using the framework implemented.

References

[1] Mnih et al. Playing Atari with Deep Reinforcement Learning. Proc. of NIPS Workshop on Deep Learning, pp. 1–9, 2013.

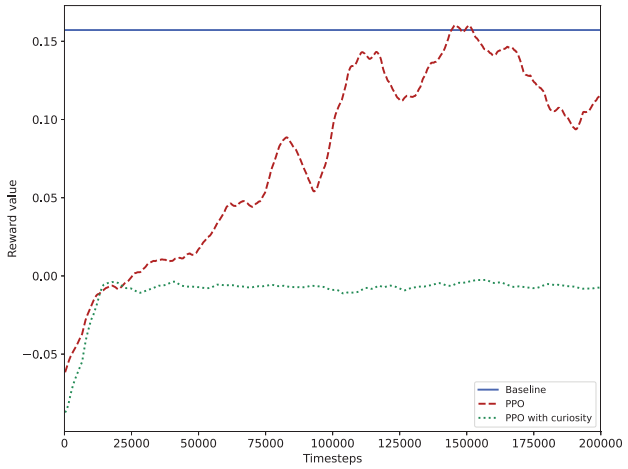
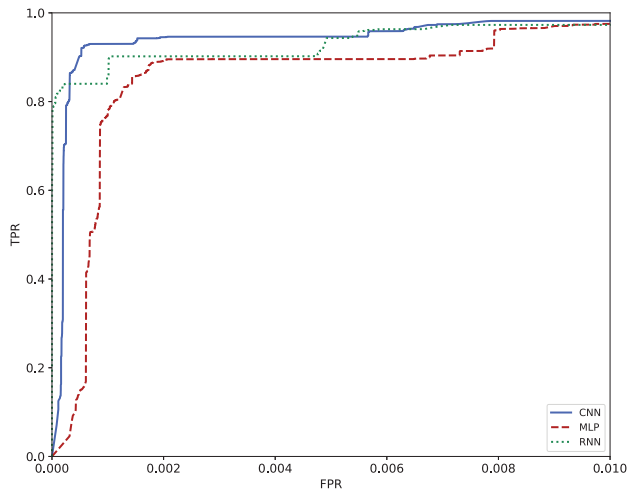


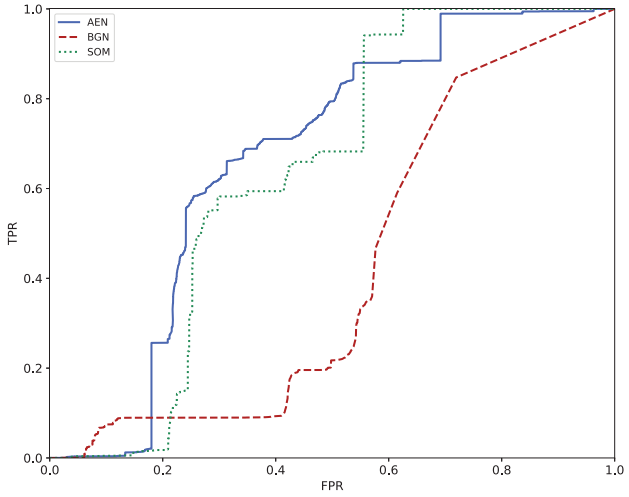
Fig. 4: Reward function during the training in case of Web attack mitigation.

[2] V. Mnih et al. Human-level control through deep reinforcement learning. Nature, vol. 518, no. 7540, pp. 529–533, 2015.
[3] D. Silver et al. Reward is enough. Artificial Intelligence, vol. 299, 2021.
[4] J. Schulman et al. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.
[5] T. Théate et al. An application of deep reinforcement learning to algorithmic trading. Expert Systems with Applications, vol. 173, 2021.
[6] M. Fu et al. Deep Reinforcement Learning Framework for Category-Based Item Recommendation. IEEE Transactions on Cybernetics, 2021.
[7] G. Sctlerhoet et al. Deep Reinforcement Learning for Industrial Insertion Tasks with Visual Inputs and Natural Rewards. arXiv:1906.05841, 2019.
[8] A. Kendall et al. Learning to Drive in a Day. arXiv:1807.00412, 2018.
[9] X. Lu et al. Reinforcement learning based PHY authentication for VANETs. IEEE Tran. on Veh. Tech., vol. 69, no. 3, pp. 3068–3079, 2020.
[10] Y.-R. Chen et al. RL-Routing: An SDN Routing Algorithm Based on Deep Reinforcement Learning. IEEE Tran. on Network Science and Engineering, vol. 7, no. 4, pp. 3185–3199, 2020.
[11] J. Rischke et al. QR-SDN: Towards Reinforcement Learning States, Actions, and Rewards for Direct Flow Routing in Software-Defined Networks. IEEE Access, vol. 8, pp. 174773–174791, 2020.
[12] et al. Optimal VNF Placement via Deep Reinforcement Learning in SDN/NFV-Enabled Networks. IEEE Journal on Selected Areas in Communications, vol. 38, no. 2, pp. 263–278, 2020.
[13] D. Lee et al. Q-learning based Service Function Chaining using VNF Resource-aware Reward Model. Proc. of the 21st Asia-Pacific Network Operations and Management Symposium, pp. 279–282, 2020.
[14] Z. Ning et al. Deep Reinforcement Learning for NFV-based Service Function Chaining in Multi-Service Networks. Invited Paper at IEEE 21st Int. Conf. on High Performance Switching and Routing, pp. 1–6, 2020.
[15] M. Zolotukhin et al. On Artificial Intelligent Malware Tolerant Networking for IoT. Proc. of NFV-SDN, pp. 1–6, 2018.
[16] M. Zolotukhin et al. Reinforcement Learning for Attack Mitigation in SDN-enabled Networks. Proc. of NetSoft, pp. 282–286, 2020.
[17] D. Pathak et al. Curiosity-driven Exploration by Self-supervised Prediction. arXiv: 1705.05363, 2017.
[18] V. Mnih et al. Asynchronous Methods for Deep Reinforcement Learning. arXiv: 1602.01783, 2016.
[19] J. Schulman. Trust Region Policy Optimization. arXiv: 1502.05477, 2017.
[20] I. Sharafaldin et al. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. Proc of the 4-th Int. Conf. on Information Systems Security and Privacy, pp. 108–116, 2018.
[21] G. Brockman et al. OpenAI Gym. arXiv:1606.01540, 2016.
[22] P. Dhariwal et al. OpenAI Baselines. GitHub, https://github.com/openai/baselines, 2017.
[23] M. Zolotukhin. RL-based SFC prototype. GitHub, https://github.com/mizolotu/izi, 2021.

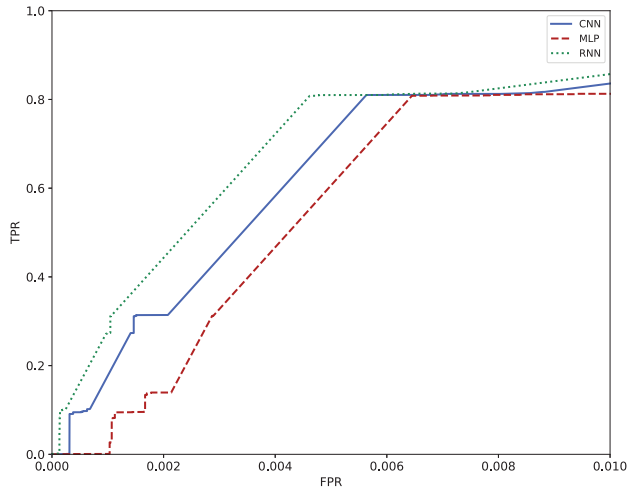
Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:06 UTC from IEEE Xplore. Restrictions apply.



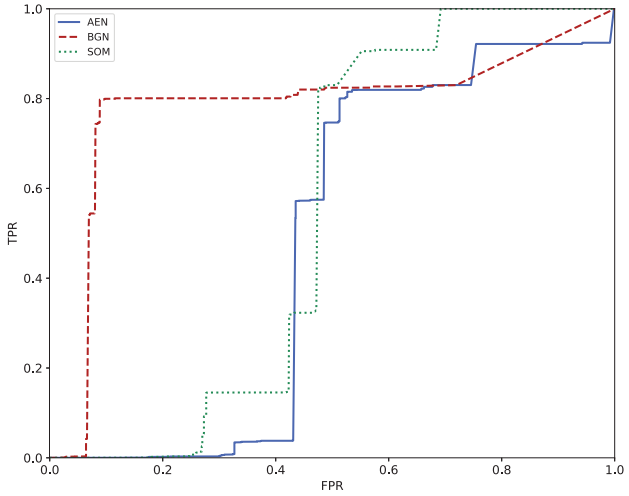
(a) DDoS (supervised).



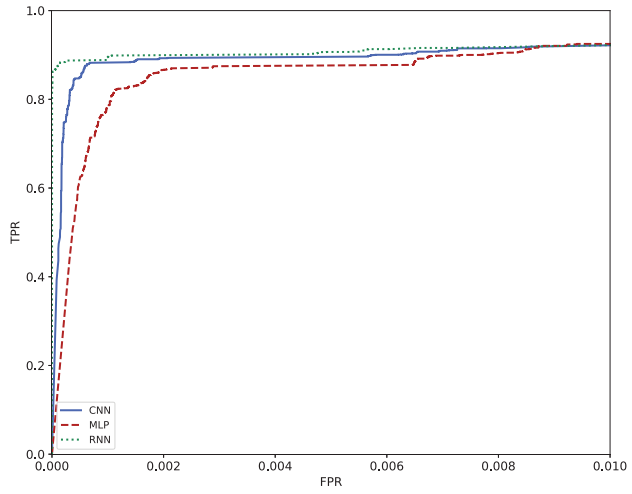
(b) DDoS (unsupervised).



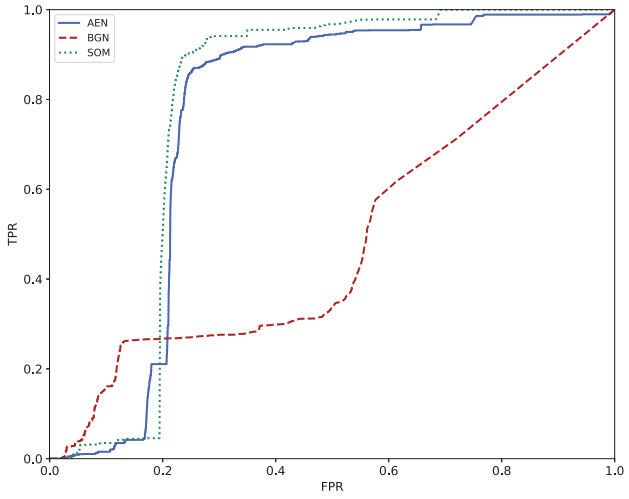
(c) Botnet (supervised).



(d) Botnet (unsupervised).



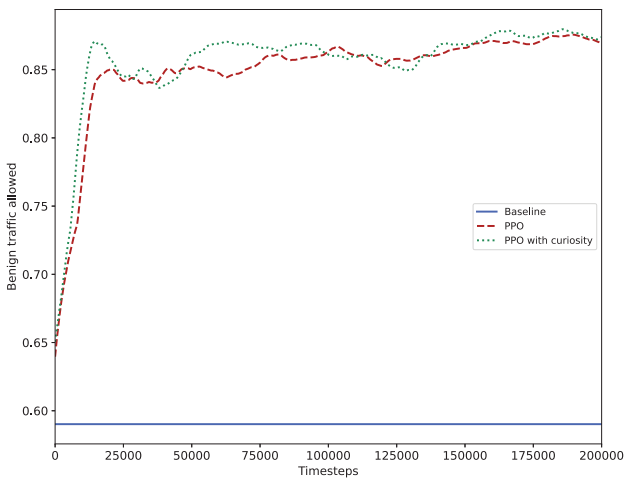
(e) Web (supervised).



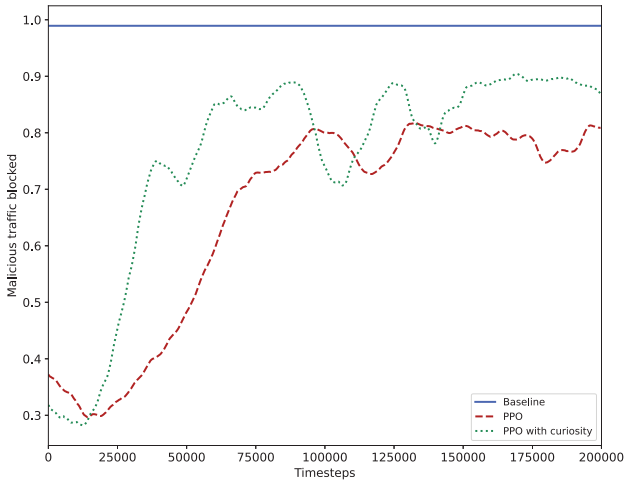
(f) Web (unsupervised).

Fig. 5: Dependence of TPR on FPR when detecting intrusions with security VNFs implemented.

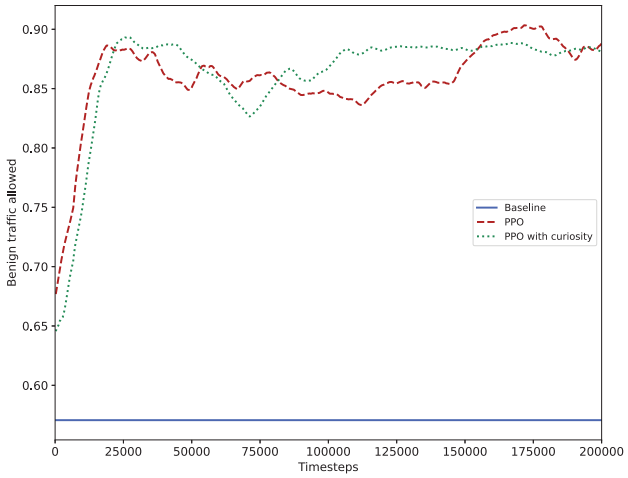
Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:06 UTC from IEEE Xplore. Restrictions apply.



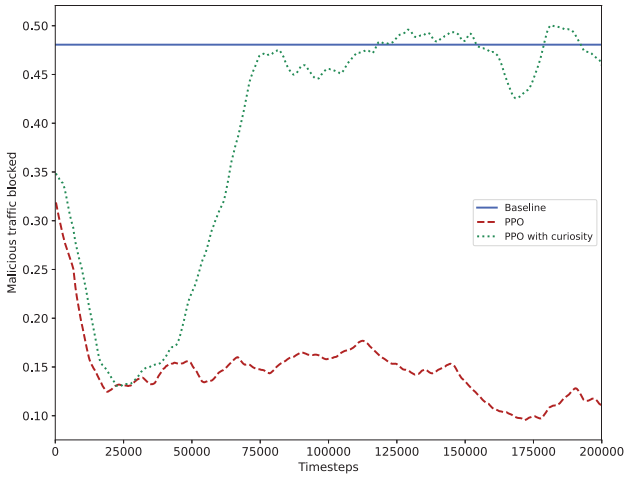
(a) DDoS (legitimate).



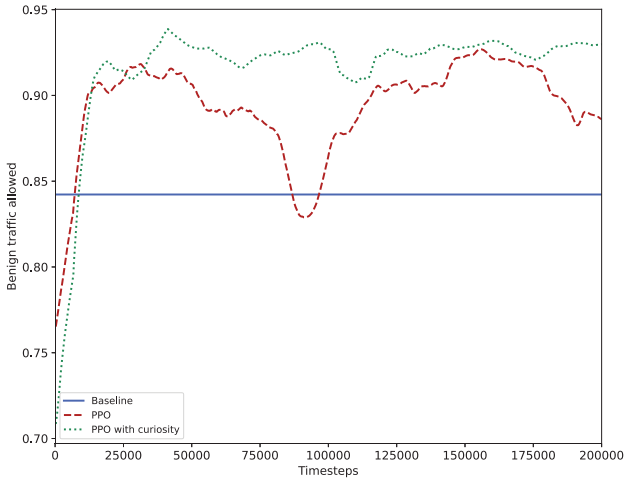
(b) DDoS (malicious).



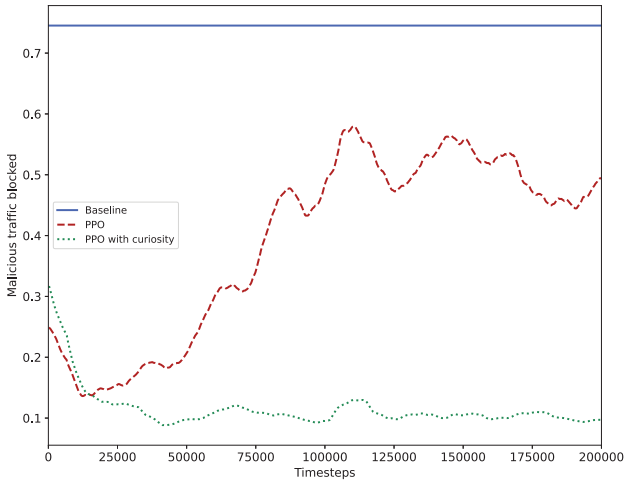
(c) Botnet (legitimate).



(d) Botnet (malicious).



(e) Web (legitimate).



(f) Web (malicious).

Fig. 6: Percentage of the legitimate traffic passed and malicious traffic blocked during the RL agent training.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:56:06 UTC from IEEE Xplore. Restrictions apply.